

Timeline Workflow Anti-Conflict + Panduan Postman

Project Sistem Reservasi Ruang - 1 Branch Main

Batas penting

- Tidak ada workflow 1 branch yang benar-benar 100% bebas error. Ini adalah workflow paling rendah risiko untuk tim yang belum terbiasa branch.
- Aturan utama: satu file hanya dipegang satu orang pada satu waktu.
- Endpoint API di dokumen ini adalah target kerja. Sebelum step terkait selesai, hasil 404 di Postman adalah normal.

Aturan Kerja 1 Branch Main

1. Sebelum mulai coding: buka VS Code, Source Control, klik Sync Changes atau Pull.
2. Tulis di grup: nama + file yang akan dikerjakan.
3. Jangan edit file yang sedang dipegang anggota lain.
4. Commit kecil setelah satu file atau satu endpoint selesai.
5. Sebelum push: run app, test endpoint sendiri di Postman, lalu Pull/Sync dulu.
6. Kalau Pull membuat conflict: stop, jangan asal Accept All. Hubungi pemilik file.
7. Setelah push: tulis di grup file apa yang berubah dan endpoint apa yang sudah bisa dites.

Cara GitHub dan VS Code GUI

1. Owner buat repo GitHub, lalu invite 5 anggota lewat Settings > Collaborators.
2. Anggota accept invite dari email atau GitHub notification.
3. Di VS Code: Source Control > Clone Repository > paste URL GitHub > pilih folder.
4. Setiap mulai kerja: Source Control > tiga titik > Pull, atau klik Sync Changes.
5. Setelah edit: klik plus pada file, tulis commit message, klik Commit.
6. Setelah commit: klik Sync Changes atau Push.
7. Jika Push ditolak: Pull dulu, selesaikan conflict, test ulang, Push lagi.

Setup Postman Dasar

Buat Environment bernama Reservasi Local:

Variable	Value
baseUrl	http://localhost:8080
csrf	diisi otomatis dari GET /login
role	mhs / adm / stm sesuai akun yang sedang dites

GET Login Page

Item	Isi
Method	GET
URL	{{baseUrl}}/login
Login role	belum login
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	200 OK, HTML login muncul, ada input _csrf.
Jika gagal	Jika 404: aplikasi belum jalan. Jika connection refused: port 8080 belum aktif.

Script tab Tests untuk GET /login:

```
const html = pm.response.text();
const match = html.match(/name="_csrf"[^>]*value="([^"]+)"/);
if (match) pm.environment.set("csrf", match[1]);
pm.test("CSRF token ditemukan", function () {
  pm.expect(pm.environment.get("csrf")).to.not.be.empty;
});
```

POST Login

Item	Isi
Method	POST
URL	{{baseUrl}}/login
Login role	mhs / adm / stm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	x-www-form-urlencoded: username=mhs, password=mhs, _csrf={{csrf}}
Expected	302 ke /dashboard atau 200 HTML jika Postman mengikuti redirect. Cookie JSESSIONID tersimpan.
Jika gagal	Jika 403: ambil CSRF ulang. Jika /login?error: username/password salah.

Timeline Utama Per File

1. Semua anggota

Apa yang dilakukan: Setup repo, clone, Pull/Push percobaan, dan cek aplikasi bisa jalan.

Kenapa dilakukan sekarang: Semua orang harus mulai dari file yang sama agar tidak ada edit di versi lama.

Yang diharapkan setelah selesai: Semua laptop punya project yang sama, app bisa dibuka, login starter bisa dites.

Urutan file yang dikerjakan:

1. README.md - baca aturan starter.
2. DEVELOPMENT_MAP.md - baca pembagian file.
3. src/main/resources/application.properties - cek database dan user MySQL.
4. src/main/resources/templates/login.html - cek halaman login starter.

Test Postman:

Cek Login Page

Item	Isi
Method	GET
URL	{{baseUrl}}/login
Login role	belum login
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	200 OK, HTML login muncul.
Jika gagal	Cek app sudah run dan port 8080 tidak dipakai aplikasi lain.

Starter code contoh:

```
// Tidak ada starter code baru pada step ini.  
// Fokus: clone, run, login page terbuka, dan semua anggota paham aturan main.
```

Hasil step 1: Tim punya base project yang sama dan siap mulai kerja per file.

2. Amelia Sofiana Makharomi

Apa yang dilakukan: Rapikan pondasi login, role, dan user starter.

Kenapa dilakukan sekarang: Semua fitur berikutnya butuh identitas user dan role yang jelas.

Yang diharapkan setelah selesai: mhs, adm, dan stm bisa login. Tiap role hanya masuk dashboard sendiri.

Urutan file yang dikerjakan:

1. auth/LoginUser.java
2. auth/UserRole.java
3. auth/LoginUserRepository.java
4. auth/DatabaseUserDetailsService.java
5. auth/StarterUserSeeder.java
6. config/SecurityConfig.java
7. controller/HomeController.java
8. templates/login.html
9. templates/mahasiswa/dashboard.html
10. templates/admin/dashboard.html
11. templates/satpam/dashboard.html

Test Postman:

Login Mahasiswa

Item	Isi
Method	POST
URL	{{baseUrl}}/login
Login role	mhs

Item	Isi
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	x-www-form-urlencoded: username=mhs, password=mhs, _csrf={{csrf}}
Expected	Redirect ke /dashboard lalu /mahasiswa/dashboard.
Jika gagal	403 berarti CSRF salah. /login?error berarti akun salah.

Cegah Akses Role Lain

Item	Isi
Method	GET
URL	{{baseUrl}}/admin/dashboard
Login role	login sebagai mhs
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	403 Forbidden.
Jika gagal	Jika 200, aturan role di SecurityConfig salah.

Starter code contoh:

```
// LoginUser: simpan username, passwordHash, role.
// UserRole: MAHASISWA, ADMIN, SATPAM.
// SecurityConfig: atur /mahasiswa/**, /admin/**, /satpam/** sesuai role.
```

Hasil step 2: Aplikasi punya autentikasi awal yang bisa dipakai semua anggota saat test.

3. Amelia Sofiana Makharomi

Apa yang dilakukan: Buat domain User dan Mahasiswa final, profil, dan DTO user.

Kenapa dilakukan sekarang: Reservation membutuhkan Mahasiswa. Jadi data Mahasiswa harus stabil dulu.

Yang diharapkan setelah selesai: Data User/Mahasiswa bisa dibuat, dibaca, dan diubah tanpa passwordHash bocor.

Urutan file yang dikerjakan:

1. entity/User.java
2. entity/Mahasiswa.java
3. dto/RegistrationRequest.java
4. dto/ProfileUpdateRequest.java
5. dto/UserResponse.java
6. repository/UserRepository.java
7. service/UserService.java
8. controller/rest/UserRestController.java
9. controller/MahasiswaController.java
10. templates/users/profile.html
11. entity/UserTests.java, entity/MahasiswaTests.java, service/UserServiceTests.java

Test Postman:

Register Mahasiswa

Item	Isi
Method	POST
URL	{{baseUrl}}/api/users/register
Login role	adm atau public sesuai keputusan tim
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"nama":"Budi","email":"budi@mail.test","noHp":"0811","password":"budi123","role":"MAHASISWA","nim":"23001","passwordHash":"TI","angkatan":2023}
Expected	201 Created, response tanpa passwordHash.
Jika gagal	400 jika field kosong. 409 jika email/nim sudah dipakai.

Update Profil

Item	Isi
Method	PUT
URL	{{baseUrl}}/api/users/profile
Login role	user login
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"nama":"Budi Update","email":"budi@mail.test","noHp":"0812","prodi":"TI","angkatan":2023}
Expected	200 OK, data profil berubah.
Jika gagal	401/403 jika belum login atau role tidak sesuai.

Starter code contoh:

```
@Entity
public class Mahasiswa extends User {
    // TODO: nim, prodi, angkatan.
    // TODO: ajukanReservasi(), batalkanReservasi(), lihatStatusReservasi().
}
```

Hasil step 3: User dan Mahasiswa siap dipakai Reservation.

4. Ali Abdul Fattah 'Alim Kautsar

Apa yang dilakukan: Buat domain Room dan API pengelolaan ruangan.

Kenapa dilakukan sekarang: Reservation tidak bisa dibuat sebelum data Room tersedia.

Yang diharapkan setelah selesai: Room bisa dibuat, dilihat, dicari, dan diaktifkan/nonaktifkan.

Urutan file yang dikerjakan:

1. entity/Room.java
2. dto/RoomRequest.java
3. dto/RoomResponse.java
4. dto/RoomStatusRequest.java
5. repository/RoomRepository.java
6. service/RoomService.java
7. controller/rest/RoomRestController.java
8. templates/rooms/list.html
9. entity/RoomTests.java, repository/RoomRepositoryTests.java

Test Postman:

Tambah Room

Item	Isi
Method	POST
URL	{{baseUrl}}/api/rooms
Login role	adm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"namaRuang":"Lab 1","gedung":"A","kapasitas":30,"statusAktif":true}
Expected	201 Created, response berisi roomId dan data room.
Jika gagal	400 jika kapasitas <= 0. 403 jika bukan admin.

Ubah Status Room

Item	Isi
Method	PATCH
URL	{{baseUrl}}/api/rooms/{roomId}/status
Login role	adm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"statusAktif":false}
Expected	200 OK, statusAktif berubah.
Jika gagal	404 jika roomId tidak ada. 409 jika ada reservasi aktif.

Starter code contoh:

```
@Entity
public class Room {
    // TODO: roomId, namaRuang, gedung, kapasitas, statusAktif.
    // TODO: aktifkan(), nonaktifkan(), ubahStatusAktif(), getInfoRuang().
}
```

Hasil step 4: Data ruangan siap dipakai oleh fitur reservasi.

5. 'Aarif Rahmaan Jalaluddin Faqiih + Ali Abdul Fattah 'Alim Kautsar

Apa yang dilakukan: Buat model Reservation, status, dan DTO kontrak reservasi.

Kenapa dilakukan sekarang: Approval dan AccessRecord bergantung pada Reservation. Modelnya harus jelas dulu.

Yang diharapkan setelah selesai: Struktur Reservation lengkap sesuai class diagram dan siap dipakai service.

Urutan file yang dikerjakan:

1. enums/ReservationStatus.java
2. entity/Reservation.java
3. dto/ReservationRequest.java
4. dto/ReservationResponse.java
5. dto/ReservationAvailabilityRequest.java
6. dto/ReservationCancelRequest.java
7. entity/ReservationTests.java

Test Postman:

Cek Kontrak Body Reservasi

Item	Isi
Method	POST
URL	{{baseUrl}}/api/reservations/availability
Login role	mhs
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"roomId":"R001","tanggal":"2026-06-10","jamMulai":"09:00","jamSelesai":"11:00"}
Expected	Jika service belum selesai: 404 normal. Jika sudah selesai: 200 OK available true/false.
Jika gagal	400 jika format tanggal/jam salah.

Starter code contoh:

```
@Entity
public class Reservation {
    // TODO: reservationId, mahasiswa, room, tanggal, jamMulai, jamSelesai.
    // TODO: tujuan, status, createdAt, cancelledAt, approval, accessRecord.
}
```

Hasil step 5: Kontrak data reservasi tidak berubah-ubah saat service mulai dibuat.

6. Ali Abdul Fattah 'Alim Kautsar

Apa yang dilakukan: Buat ReservationService, repository, REST, dan halaman form/list reservasi.

Kenapa dilakukan sekarang: Setelah User, Mahasiswa, dan Room siap, fitur reservasi bisa dibuat aman.

Yang diharapkan setelah selesai: Mahasiswa bisa cek ketersediaan, membuat reservasi, melihat riwayat, dan membatalkan.

Urutan file yang dikerjakan:

1. repository/ReservationRepository.java
2. service/ReservationService.java
3. controller/rest/ReservationRestController.java
4. templates/reservations/form.html
5. templates/reservations/list.html
6. repository/ReservationRepositoryTests.java
7. service/ReservationServiceTests.java

Test Postman:

Cek Availability

Item	Isi
Method	POST
URL	{{baseUrl}}/api/reservations/availability
Login role	mhs
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"roomId":"R001","tanggal":"2026-06-10","jamMulai":"09:00","jamSelesai":"11:00"}
Expected	200 OK, data.available true jika kosong.
Jika gagal	409 jika bentrok, 400 jika jam tidak valid, 404 jika room tidak ada.

Ajukan Reservasi

Item	Isi
Method	POST
URL	{{baseUrl}}/api/reservations
Login role	mhs
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"roomId":"R001","tanggal":"2026-06-10","jamMulai":"09:00","jamSelesai":"11:00","tujuan":"Rapat UKM"}
Expected	201 Created, status awal sesuai ReservationStatus.
Jika gagal	403 jika bukan mahasiswa. 409 jika jadwal bentrok.

Batalan Reservasi

Item	Isi
Method	PATCH
URL	{{baseUrl}}/api/reservations/{reservationId}/cancel
Login role	mhs pemilik reservasi
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"alasanPembatalan":"Jadwal berubah"}
Expected	200 OK, cancelledAt terisi dan status berubah.
Jika gagal	403 jika bukan pemilik. 409 jika isCanBeCancelled() false.

Starter code contoh:

```
@RestController
@RequestMapping("/api/reservations")
public class ReservationRestController {
    // TODO: endpoint availability, create, history, cancel.
}
```

Hasil step 6: Alur pengajuan reservasi selesai dan bisa diuji sebelum approval dibuat.

7. Atha Muyassar

Apa yang dilakukan: Buat Admin dan kontrak Notifiable untuk penerima notifikasi.

Kenapa dilakukan sekarang: Approval membutuhkan Admin sebagai pihak yang memberi keputusan.

Yang diharapkan setelah selesai: Admin punya data unitKerja dan siap dipakai Approval.

Urutan file yang dikerjakan:

1. entity/Admin.java
2. entity/Notifiable.java
3. dto/AdminRequest.java
4. controller/AdminController.java
5. controller/rest/AdminRestController.java
6. templates/admin/dashboard.html
7. entity/AdminTests.java

Test Postman:

Cek Profil Admin

Item	Isi
Method	GET
URL	{{baseUrl}}/api/admin/profile
Login role	adm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	200 OK, berisi data Admin dan unitKerja.
Jika gagal	403 jika bukan admin. 404 jika endpoint belum dibuat.

Starter code contoh:

```
public class Admin extends User implements Notifiable {  
    // TODO: unitKerja.  
    // TODO: verifikasiReservasi(), setujuReservasi(), tolakReservasi().  
}
```

Hasil step 7: Fitur approval punya aktor Admin yang jelas.

8. 'Aarif Rahmaan Jalaluddin Faqiih

Apa yang dilakukan: Buat Approval, ApprovalService, REST approval, dan halaman approval.

Kenapa dilakukan sekarang: Approval baru aman dibuat setelah Reservation dan Admin siap.

Yang diharapkan setelah selesai: Admin bisa setuju, tolak, atau minta revisi reservasi.

Urutan file yang dikerjakan:

1. enums/ApprovalDecision.java
2. entity/Approval.java
3. dto/ApprovalRequest.java
4. dto/ApprovalResponse.java
5. repository/ApprovalRepository.java
6. service/ApprovalService.java
7. controller/rest/ApprovalRestController.java
8. templates/approvals/list.html
9. entity/ApprovalTests.java, service/ApprovalServiceTests.java

Test Postman:

Lihat Pending Approval

Item	Isi
Method	GET
URL	{{baseUrl}}/api/approvals/pending
Login role	adm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	200 OK, list reservasi menunggu approval.
Jika gagal	403 jika bukan admin.

Approve Reservasi

Item	Isi
Method	POST
URL	{{baseUrl}}/api/approvals/{reservationId}/approve
Login role	adm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"catatan":"Disetujui"}
Expected	200 OK, Approval terbentuk, status Reservation berubah disetujui.

Item	Isi
Jika gagal	409 jika status reservasi tidak bisa di-approve.

Reject Reservasi

Item	Isi
Method	POST
URL	{{baseUrl}}/api/approvals/{reservationId}/reject
Login role	adm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"catatan":"Jadwal tidak sesuai"}
Expected	200 OK, Approval terbentuk, status Reservation berubah ditolak.
Jika gagal	400 jika catatan kosong saat reject.

Starter code contoh:

```
@Service
public class ApprovalService {
    // TODO: verifyReservation(), approveReservation(), rejectReservation(), requestRevision().
}
```

Hasil step 8: Alur keputusan reservasi selesai dan status reservasi sudah terkendali.

9. Tadzkiroh Aziziyah Haqia

Apa yang dilakukan: Buat Satpam dan AccessRecord.

Kenapa dilakukan sekarang: Check-in/check-out butuh data Satpam dan Reservation yang sudah disetujui.

Yang diharapkan setelah selesai: AccessRecord punya data checkInTime, checkOutTime, dan catatan kendala.

Urutan file yang dikerjakan:

1. entity/Satpam.java
2. entity/AccessRecord.java
3. dto/SatpamRequest.java
4. dto/AccessIssueRequest.java
5. dto/AccessRecordResponse.java
6. repository/AccessRecordRepository.java
7. controller/SatpamController.java
8. entity/SatpamTests.java, entity/AccessRecordTests.java

Test Postman:

Cek Kontrak AccessRecord

Item	Isi
Method	GET
URL	{{baseUrl}}/api/access
Login role	stm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	Jika service belum selesai: 404 normal. Jika sudah selesai: 200 OK list AccessRecord.
Jika gagal	403 jika bukan satpam.

Starter code contoh:

```
public class AccessRecord {
    // TODO: recordId, reservation, satpam, checkInTime, checkOutTime.
    // TODO: checkIn(), checkOut(), laporkanKendala().
}
```

Hasil step 9: Data akses siap dipakai oleh service check-in/check-out.

10. Tadzkiroh Aziziyah Haqia

Apa yang dilakukan: Buat AccessControlService, REST access, dan halaman akses.

Kenapa dilakukan sekarang: Setelah Approval selesai, satpam bisa memproses akses ruangan.

Yang diharapkan setelah selesai: Satpam bisa check-in, check-out, dan melaporkan kendala.

Urutan file yang dikerjakan:

1. service/AccessControlService.java
2. controller/rest/AccessRestController.java
3. templates/access/list.html
4. repository/AccessRecordRepositoryTests.java
5. service/AccessControlServiceTests.java

Test Postman:

Check-In

Item	Isi
Method	POST
URL	{{baseUrl}}/api/access/check-in
Login role	stm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"reservationId":"RSV001"}
Expected	200 OK, AccessRecord berisi checkInTime.
Jika gagal	409 jika reservasi belum disetujui atau sudah check-in.

Check-Out

Item	Isi
Method	POST
URL	{{baseUrl}}/api/access/check-out
Login role	stm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"reservationId":"RSV001"}
Expected	200 OK, AccessRecord berisi checkOutTime.
Jika gagal	409 jika belum check-in atau sudah check-out.

Lapor Kendala

Item	Isi
Method	POST
URL	{{baseUrl}}/api/access/issues
Login role	stm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"reservationId":"RSV001","catatanPelanggaran":"Kunci ruangan terlambat dikembalikan"}
Expected	200 OK, catatanPelanggaran tersimpan.
Jika gagal	400 jika catatan kosong.

Starter code contoh:

```
@Service
public class AccessControlService {
    // TODO: checkIn(), checkOut(), reportIssue().
}
```

Hasil step 10: Alur pemakaian ruangan setelah approval selesai.

11. Ajda Mutiara Zahra

Apa yang dilakukan: Buat Notification dan NotificationService.

Kenapa dilakukan sekarang: Notifikasi paling aman dibuat setelah status Reservation, Approval, dan AccessRecord jelas.

Yang diharapkan setelah selesai: User/role terkait menerima notifikasi status dan bisa tandai dibaca.

Urutan file yang dikerjakan:

1. entity/Notification.java
2. dto/NotificationRequest.java
3. dto/NotificationResponse.java
4. repository/NotificationRepository.java
5. service/NotificationService.java
6. controller/rest/NotificationRestController.java
7. templates/notifications/list.html
8. entity/NotificationTests.java, service/NotificationServiceTests.java

Test Postman:

List Notification

Item	Isi
Method	GET
URL	{{baseUrl}}/api/notifications
Login role	mhs / adm / stm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	200 OK, list notifikasi milik user login.
Jika gagal	401 jika belum login.

Send Notification

Item	Isi
Method	POST
URL	{{baseUrl}}/api/notifications/send
Login role	adm atau internal sesuai keputusan tim
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	{"penerimaId":"U001","penerimaRole":"MAHASISWA","pesan":"Reservasi disetujui"}
Expected	201 Created, statusBaca false, createdAt terisi.
Jika gagal	404 jika penerima tidak ada. 400 jika pesan kosong.

Tandai Dibaca

Item	Isi
Method	PATCH
URL	{{baseUrl}}/api/notifications/{notificationId}/read
Login role	pemilik notifikasi
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	200 OK, statusBaca menjadi true.
Jika gagal	403 jika notifikasi milik user lain.

Starter code contoh:

```
@Service
public class NotificationService {
    // TODO: sendNotification(), sendStatusUpdate().
    // TODO: panggil Notification.tandaiDibaca().
}
```

Hasil step 11: Alur pemberitahuan selesai dan bisa dipasang ke approval/reservasi/access.

12. Semua anggota

Apa yang dilakukan: Integrasi halaman, layout, CSS, error handler, dan test gabungan.

Kenapa dilakukan sekarang: Setelah semua fitur inti ada, barulah UI dan integrasi disatukan.

Yang diharapkan setelah selesai: Menu role rapi, error jelas, dan semua endpoint utama lolos Postman.

Urutan file yang dikerjakan:

1. templates/layout/main.html
2. templates/fragments/sidebar.html
3. templates/error.html
4. static/css/style.css
5. exception/GlobalExceptionHandler.java
6. dto/ApiResponse.java
7. dto/ApiErrorResponse.java
8. GlobalExceptionHandlerTests.java
9. LoginSecurityTests.java
10. ReservasiApplicationTests.java

Test Postman:

Regression Role

Item	Isi
Method	GET
URL	{{baseUrl}}/mahasiswa/dashboard, /admin/dashboard, /satpam/dashboard
Login role	tes bergantian mhs, adm, stm
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Tidak ada body.
Expected	Role sendiri 200 OK, role lain 403 Forbidden.
Jika gagal	Jika role lain 200, cek SecurityConfig.

Regression API Utama

Item	Isi
Method	GET/POST/PATCH
URL	rooms, reservations, approvals, access, notifications
Login role	sesuai endpoint
Headers	Content-Type: application/json. Untuk POST/PATCH, tambahkan X-CSRF-TOKEN: {{csrf}} bila CSRF aktif.
Body	Pakai body dari step masing-masing.
Expected	Semua response sesuai status code yang disepakati.
Jika gagal	Catat endpoint gagal, pemilik file memperbaiki sebelum push berikutnya.

Starter code contoh:

```
// ApiResponse: status, message, data.  
// ApiErrorResponse: status, message, path, timestamp.  
// Sidebar: tampilkan menu sesuai role login.
```

Hasil step 12: Project siap demo internal dan bug sudah dipetakan per pemilik file.

Urutan Endpoint Postman Ringkas

Urutan	Endpoint	Role	Tujuan
1	GET /login + POST /login	mhs/adm/stm	ambil session login
2	POST /api/users/register	adm/public	buat user data awal
3	POST /api/rooms	adm	buat room
4	POST /api/reservations/availability	mhs	cek jadwal kosong
5	POST /api/reservations	mhs	ajukan reservasi
6	GET /api/approvals/pending	adm	lihat pending
7	POST /api/approvals/{id}/approve	adm	setujui
8	POST /api/access/check-in	stm	mulai pemakaian
9	POST /api/access/check-out	stm	selesai pemakaian
10	GET /api/notifications	mhs/adm/stm	cek notifikasi

Skenario Jika Gagal di Tengah Jalan

Masalah	Penyebab Umum	Tindakan Cepat
Push ditolak	Main sudah berubah	Klik Pull/Sync Changes di VS Code, test ulang, lalu Push.
Conflict	Dua orang edit file sama	Jangan asal Accept All. Hubungi pemilik file, buka Merge Editor.
App tidak jalan	Compile error dari file terakhir	Pemilik push terakhir wajib fix dulu. Anggota lain jangan push fitur baru.
Postman 401	Belum login/session hilang	GET /login lagi, POST /login lagi, cek cookie JSESSIONID.
Postman 403	Role salah atau CSRF salah	Login dengan role sesuai endpoint. Ambil CSRF ulang.
Postman 404	Endpoint belum dibuat atau URL salah	Cek step terkait sudah selesai. Cek path controller.
Postman 400	Body JSON salah	Cek Content-Type application/json dan nama field sesuai DTO.
Postman 409	Bentrok aturan bisnis	Cek room aktif, jadwal bentrok, status reservasi, atau record sudah ada.
Database tidak muncul	App belum run atau koneksi MySQL salah	Jalankan MySQL, cek username/password, run app sampai Hibernate membuat tabel.

Aturan Pencegahan Conflict

1. File owner tetap: Amelia User/Mahasiswa/Auth, Atha Admin/Notifiable, Tadzkiroh Satpam/Access, Ali Room/ReservationService, 'Aarif Reservation/Approval, Ajda Notification.
2. File bersama seperti SecurityConfig, layout, sidebar, ApiResponse, GlobalExceptionHandler hanya diedit saat integrasi dan diumumkan dulu.
3. Jangan commit folder target, file .class, atau hasil build.
4. Jangan format seluruh project saat orang lain sedang mengerjakan.
5. Jika endpoint butuh file orang lain, tulis TODO atau koordinasi. Jangan ambil alih file tanpa izin.
6. Setiap selesai step, pemilik menjalankan test Postman step itu dan menulis hasil di grup.

Checklist Selesai Harian

1. Pull/Sync sebelum mulai.
2. Kerjakan file milik sendiri.
3. Run aplikasi.
4. Test endpoint Postman milik sendiri.
5. Commit dengan pesan jelas.
6. Pull/Sync lagi.
7. Jika aman, Push.
8. Tulis update di grup: file berubah, endpoint berhasil, kendala bila ada.